

STUDENT 4

LLM Decision Viewer, Integration and Documentation

Show what the LLM decided and why, help the team integrate, and make the project easy to run.

ABMS-LLM Cyber Organizational Simulation

Beginner work plan for implementing LLM agents in the current repository. This document is written for both the assigned student and an AI coding assistant.

Main research question

How does agent behavior and performance differ when LLM-driven cyber agents are organized as an independent swarm versus a centralized hierarchy?

Important decision

- LLM cognition is now the main implementation target.
- The existing rule-based cognition stays small and working only as a baseline and API-failure fallback.
- Do not delete the rule baseline, because the research needs a control and developers need a cheap debugging mode.
- Work in the Conda environment named mesa and use Mesa 3.x APIs.

1. Your Job in Simple Words

You own the Solara interface, visual explanation, documentation and final integration checklist. You display safe summaries of LLM decisions and metrics. You never put simulation rules or API keys in the user interface.

How one LLM decision works

- 1 Student 1 turns the current simulation state into a small observation dictionary.
- 2 Student 2 sends that observation to the LLM with instructions and allowed actions.
- 3 The LLM returns one JSON decision. Student 2 checks it before trusting it.
- 4 Student 1's resolver applies the valid action and returns the result.
- 5 Student 3 records behavior, outcome, tokens, time, errors and estimated cost.
- 6 Student 4 shows safe decision summaries and metrics in the UI and documentation.

Words you need to know

Word	Simple meaning
LLM	A language model that reads an observation and chooses an action.
API	The online service used by Python to ask the LLM for a response.
Prompt	The instructions and simulation information sent to the LLM.
JSON	A strict text format used so Python can safely read the LLM answer.
Token	A small unit of text used to measure API usage and cost.
Fallback	A safe rule-based action used when the API or LLM answer fails.
Mock client	A fake API used in tests so tests are fast, free and repeatable.

2. Files You Own

File	What it should do
abms_cyber/visualization/app.py	Controls, organization/model selection and page layout.
abms_cyber/visualization/network_view.py	Network and agent positions.
abms_cyber/visualization/portrayal.py	Colors and role markers.
abms_cyber/visualization/charts.py	Simulation and LLM usage charts.
abms_cyber/visualization/decision_log.py	Create a safe, readable LLM decision panel.
README.md and docs/	Beginner setup, API configuration, architecture and experiment instructions.
.env.example	Document variable names with fake placeholders only; never include a real key.

What you must build

- A UI choice for swarm or centralized mode and supported cognition mode (LLM or rule baseline).
- A decision panel showing agent/leader, step, chosen action, target, short reason, validation result and whether fallback was used.
- Charts or counters for calls, tokens, latency, errors and estimated cost when available.
- Distinct display for swarm agents, centralized leader and centralized workers.
- Beginner documentation for environment setup, API key environment variable, one simulation, one experiment and common errors.
- An integration checklist that every student can run before the final demonstration.

Stay inside your role

Do not redesign another student's layer without agreement. If you need a new field or function, first write a small example of the input and output, then ask its owner to approve the interface.

3. Shared LLM Data Format

All students should use one simple data format. Student 1 builds the observation. Student 2 sends it to the LLM and validates the response. Students 3 and 4 read safe metadata from the result.

Example observation

```
{
  "agent_id": 1,
  "organization": "swarm",
  "current_node": "3",
  "privilege_level": 0,
  "known_nodes": ["3", "4"],
  "recent_actions": ["SCAN:3"],
  "detection_level": 0.20,
  "goal": "Compromise the target with admin privilege",
  "allowed_actions": ["SCAN", "MOVE", "EXPLOIT", "ESCALATE"]
}
```

Example LLM response

```
{
  "action": "EXPLOIT",
  "target_id": "3",
  "reason": "The current node is scanned and not compromised."
}
```

Safety rules

- Never execute Python code returned by the LLM.
- Never trust action or target text until it is checked against allowed values.
- Never hardcode, commit, print or display the API key.
- Use environment variables for secrets and fake clients for normal tests.
- Keep prompts small enough to control cost and store only safe logs.

4. Build It Step by Step

- 1 Agree the safe decision-log and metric fields with Students 2 and 3. Never display a secret key or raw provider request headers.
- 2 Make app.py pass the selected organization and cognition mode into the model instead of always creating swarm.
- 3 Create decision_log.py and display the newest validated decisions in plain language.
- 4 Update portrayal so a centralized leader looks different from workers and swarm agents.
- 5 Add small charts/counters from DataCollector for LLM calls, total tokens, latency, errors and fallback.
- 6 Keep rendering read-only. Buttons may call model.step or reset; drawing code must not change the simulation.
- 7 Rewrite README as a beginner runbook: activate mesa, install dependencies, set an environment variable, run one cheap test, launch UI and run experiments.
- 8 Document cost safety, supported provider/model configuration, how to use mocked mode and how to recognize API failures.
- 9 Lead a final test of headless rule baseline, headless LLM mode, both organizations, UI step/reset and documentation commands.

When using an AI coding assistant

- Give the assistant this PDF and the repository files named in your assignment.
- Ask it to inspect before editing and to explain what already exists.
- Request one small task at a time; review the diff before moving to the next task.
- Require tests with a mock API before making a paid live request.
- Do not accept code that hides errors, exposes secrets or changes unrelated files.

5. Tests and Definition of Done

Your work is done only when these checks pass:

- The UI launches without an API key when mock or rule-baseline mode is selected.
- LLM mode reports a missing key clearly and does not show a traceback to the beginner.
- Switching organization creates the correct model and role display.
- Step changes the model once; Reset clears previous decision history.
- Decision reason and usage metrics appear without secrets or raw sensitive data.
- Empty metric data and API failures do not break charts.
- Every command in README has been run successfully from the project root.

Minimum evidence to submit

- List of files changed and one sentence explaining each file.
- Automated test output for normal, invalid and failure behavior.
- One end-to-end mocked-LLM run that costs nothing.
- One carefully limited live-LLM smoke run if the team has an API key and budget.
- A note listing remaining limitations and any assumptions agreed by the team.

Cost warning

Do not begin with many trials. Start with mocked responses, then one live step, then one short episode. Student 3 must add call/token/cost limits before the team runs large experiments.

6. Six-Week Plan and Team Handoffs

Week	Main result
1	Agree safe UI fields and run every current project mode.
2	Add model/organization/cognition controls and correct reset behavior.
3	Build decision viewer, role portrayal and LLM metric displays.
4	Write beginner API setup, runbook and troubleshooting docs.
5	Lead integration test and send defects to the correct owner.
6	Polish final demonstration, screenshots and project handover.

Required team handoffs

- Student 1 gives Student 2 the observation fields and action-resolver interface.
- Student 2 gives Student 3 sanitized LLM usage events and gives Student 4 safe decision-log fields.
- Student 3 gives Student 4 stable DataCollector column names and termination status.
- Student 4 gives everyone the final runbook and reports integration defects to the correct owner.

Four contracts to agree in Week 1

- Observation JSON fields.
- LLM response/action JSON fields.
- One model-step sequence for swarm and centralized modes.
- LLM usage and decision-log fields for metrics and UI.

7. Prompt to Give an AI Coding Assistant

Attach or provide this PDF and then send the following prompt. The assistant must still inspect the real repository because this plan describes responsibilities, not the final code state.

You are helping Student 4 on AbstractCyberSim. Own visualization, safe LLM decision display, beginner documentation and integration checks. Read Knowledge/Requirement, Knowledge/Important rules, visualization files, README/docs, and the stable fields provided by Students 1-3 before editing. Use Mesa 3.x and Solara. Add organization/cognition controls, a safe decision log, LLM usage counters/charts, and distinct swarm/leader/worker portrayal. Keep visualization read-only except Step/Reset controls. Never expose API keys or sensitive request data. Write beginner commands and troubleshooting, add pure-function tests where possible, and perform a UI smoke test plus headless runs.

Ask the AI to finish each reply with

- What it changed in beginner-friendly language.
- Which files it changed.
- Which tests and commands it ran and their results.
- Any unresolved problem or decision needed from another student.
- Whether any live API call was made and its approximate usage/cost.

Final reminder

The goal is not merely to make an API call. The goal is to create a controlled research system where valid LLM decisions can be compared fairly between swarm and centralized organizations.